

24/8/26 DAY 1:

Installation & Setup of Development Environment

1. Msys2 Download

- Download Msys2
- Run the installer
- Keep the default settings
- Command prompt like window will open.
- Paste the command: **pacman -S mingw-w64-ucrt-x86_64-gcc**
- y and Enter
- Global windows search: enviroment variables
- Edit the system environment variables
- Environment Variables
- User variables
- Inside path add: C:\msys64\ucrt64\bin
- Apply and close

2. Eclipse Download

- Download Eclipse Installer
- Download C/C++ IDE
- Install
- Run Eclipse IDE

Congratulations! You have successfully installed the C/C++ Development Environment.

SDK (Software Development Kit)

Development Tools:

- Editor: notepad, notepad++, vscode, eclipse (IDE), devc++, etc.
- Compiler: gcc, g++.
- Assembler: asm
- Linker: ld
- Documentation: cppreference, tenouk, microsoft c++ docs.
- Libraries
 - standard libraries
 - external libraries
- Runtime Environment
 - c runtime

What is Programming?

- It is set of instruction given to the computer to perform a specific task to produce an expected output after execution.

History of Programming Languages

- 1940-50s : Assembly
- 1950-60s : FORTRAN
- 1960-70s : COBOL
- 1960-70s : BASIC
- 1972 : C

C language was developed to develop the UNIX OS features.

History of C

- Created : 1969-72
- Where : Bell labs
- Who : Dennis Ritchie

Standardization of C

ANSI C standardization: 1989

ISO : C90, C99, C11, C17, C23

History of C++

- 1979: Development started
- 1982: C with classes
- 1983: C++ : ++ upgraded of c, ++ is increment operator which is used to denote upgradation.
- 1985: first commercial version
- 1989: it's c++ 2.0 version
- Who: Bjarne Stroustrup
- Where: AT&T Bell Labs

Standardization of C++

ANSI C++ standardization: C++98, C++03

ISO standardization: C++11, C++14, C++17, C++20, C++23, C++26, C++29

C++ vs C

- Procedural to OOP
- Function oriented(centered) to Object & Classes
- Global to Abstraction

Features of C++

- OOP
- Portable
- High level but provide low level control
- High performance

- Memory Management
- Rich Standard Library
- Multithreaded
- Exception handling
- Generic programming

Structure of C++ program

```
#include <iostream> // include directive: iostream header file

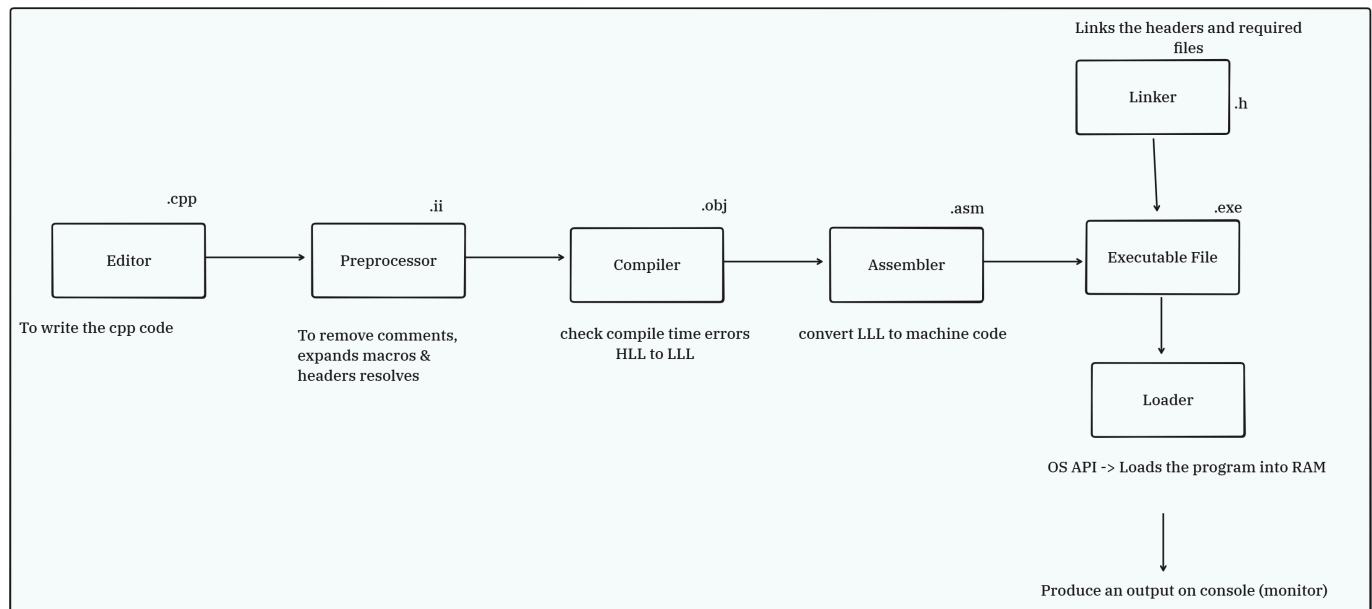
int main() // main function
{ // scope of main function starts

    std::cout << "Hello World!"; // statement to print output on console

    return 0; // it is return statement, which denotes program execution is done
or not
} // scope of main function ends here

// 0 : that this program is executed successfully
// not 0 : this program is not executed successfully
```

Execution Flow of C/C++ Program



Execution Flow of C/C++ Program

main() entry point

Main is the entry point of the program. It is the first function which is executed when the program is run.

```
int main(){}
int main(void){}
int main(int argc, char argv[]){}
```

```
int main(int argc, char argv[], char envp[]){}

//argc : arguments count
//argv : arguments given
//envp : environment parameters

int main(int argc, char** argv){}
int main(int argc, char** argv, char** envp){}

void main(){} // not recommended
```

Variables & Data Types

Variables

- A named container/entity/object which stores a value/data in a memory location.

Rules:

- cannot start with digits
- cannot start with special characters unless it is _
- no spaces in name
- cannot same as keywords

int x; it is being used to store integer values in memory location named x.

Convention:

- It is developer specific, not a standard convention of the language
- camelCase, lowercase

Data Types

- **Primitive:**

- int
- char
- float
- double
- void
- bool

Derived:

- Array
- Pointer
- function

User-Defined:

- struct
- class
- union
- enum

Type Specifier

- short : reduces the range of data type
- long : increases the range of data type
- signed : can accomodate -ve/+ve values
- unsigned can accomodate +ve values only

Type Modifiers

- const : readonly it cannot be changed
- static : one single copy for class/program
- volatile : it can change -> for hardware

Properties of Data Types:

- Range: Limit till how much data can be stored
- Size: How much memory it will take
- Nature: what type of data it represents

Example:

```
int : size 4 byte, range -2,147,483,648 to 2,147,483,647
```

16 bit, 32 bit, 64 bit architecture

Size of int in 16 bit, 32 bit and 64 bit architecture

16 bit	32 bit	64 bit
2	4	8

Range of int: $-2^{(n-1)}$ to $2^{(n-1)}-1$

Declaration vs Definition

Declaration

- It is a statement which tells the compiler about the variable name and its data type. It does not allocate memory for the variable.

```
int a; // declaration + definition
```

Definition

- It is a statement which tells the compiler about the variable name and its data type. It also allocates memory for the variable.

```
int a = 10; // definition + initialization  
int a; // declaration + definition
```

Initialization vs Assignment

Initialization

- It is a statement which assigns a value to the variable at the time of declaration.

```
int a = 10; // initialization
```

Assignment

- It is a statement which assigns a value to the variable after its declaration.

```
int a; // declaration + definition  
a = 10; // assignment
```

Comments

- Documentation purpose.
- To understand what these code does.

```
// single line comment
```

```
/*  
    block of comment/ multiline comment  
*/
```

Instead of writing comments, it is better to write self-explanatory code. Give meaningful names to variables, functions, classes, etc. so that it is easy to understand what the code does.

C++ Tokens

- Smallest unit of a program
- Keywords, Identifiers, Literals, Operators, Expressions, Statements, Punctuators

```
int main()
{
    std::cout << "Hello World!";
    int a = 10;

    return 0;
}
```

Tokens: `int`, `main`, `(`, `)`, `{`, `std`, `::`, `cout`, `<<`, `" Hello World! "`, `;`, `return`, `0`, `;`, `}`

Keywords, Identifiers and Literals

Keywords

- Inbuilt words which are reserved to the language and cannot be used by variables
- `int` `main` `if` `else` `new` `return` `class` `public` `void` `static` `const`

Identifier

- a name given to a entity/object so that it can be identified with that name.
- variable, functions, class, objects, structures

Literals

- It is a constant value which is assigned to a variable. It cannot be changed during the execution of the program.
- It is something which is written in the code and it is not a variable.
- It produces result.
- **10** : int literal
- **'A'**: char literal
- **18.6**: double literal
- **5.6f**: float literal
- **true**: bool literal
- **NULL**: Macro -> 0 int literal
- **"shil"**: string literal

Expressions

- It is a combination of operators, variables and operands
- `int result = a * b + c;`

```
a * b + c -> expression  
int result = a * b + c; -> statement
```

Statements

- It is a combination of expressions which produces a result.
- `int result = a * b + c;`

Expression vs Statement

- Expression produces a result
- Statement produces a result and it is a complete unit of execution.

Operators

Arithmetic Operator

```
- + - * / %
```

Relational Operator

```
- < > <= >= != ==
```

Logical Operator

```
- && || !
```

Unary Operator

```
- ++ -- + -
```

Ternary Operator

```
- ? :
```

Assignment Operator


```
- = += -= *= /= %=
```

Bitwise Operator

```
- & | << >>
```

Special Operator

```
- . * .* -> [] () sizeof typedef ::
```

Operator precedence and associativity

Important C++17 features

- constexpr
- filesystem
- noexcept
- inline variable
- structured bindings
- optional

LAB: Some problem-solving programs

Add two numbers

Area calculation

Swap two numbers

Calculate power

Calculate simple interest

Calculate compound interest